MLGWorks RebindX

Complete User and Integration Guide

MLGWorks

August 17, 2026

Abstract

MLGWorks RebindX is a Unity Input System asset for building reliable key-binding and controller-binding menus. It supports ordinary bindings, composite bindings, multiple profiles, versioned persistence, conflict detection and resolution, device-aware prompts, accessibility notifications, custom storage backends, generated control wrappers, and testable runtime services. This guide is written for both first-time Unity users and developers integrating RebindX into a larger production project.

Contents

1 What RebindX does

RebindX changes Unity Input System binding *overrides* at runtime. It does not rewrite the original `.inputactions` asset. The default bindings remain intact, which means a player can reset settings, a developer can update the asset, and a new profile can start from clean defaults.

The package has two runtime assemblies:

- `MLGWorks.RebindX`: core services, persistence, profiles, options, and manager APIs.
- `MLGWorks.RebindX.UI`: `RebindActionUI`, device-aware display helpers, and the custom Inspector. It is optional and requires TextMeshPro and Unity Localization in the current package setup.

The source repository also contains separate EditMode and PlayMode test assemblies. They test RebindX behavior, not `MLGWorks.Utils`.

2 Requirements and installation

2.1 Requirements

Use a Unity project with:

- Unity's Input System package enabled.
- An `InputActionAsset` or a generated `PlayerInputControls` wrapper.
- TextMeshPro and Unity Localization if you use `RebindActionUI`.
- The repository's `MLGWorks.Utils` dependency when using the source layout.

If you distribute RebindX as a package, keep one compatible Utils version in the consuming project. Do not place several different copies of the same Utils assembly in one Unity project; that causes duplicate type and assembly-resolution problems.

2.2 Installation checklist

1. Copy or install the RebindX folder under the Unity `Assets` directory.
2. Open Package Manager and install/enable Input System, TextMeshPro, and Localization as appropriate.
3. Allow Unity to finish importing and compiling assemblies.
4. Open **Window > General > Test Runner** and run the RebindX tests before integrating into a production scene.

3 The quickest working setup

3.1 1. Create an Input Actions asset

Choose **Assets > Create > Input Actions**. Create an action map, an action, and at least one binding. A minimal example is:

```
Gameplay
  Jump Button <Keyboard>/space
  Move Value 2D Vector composite
    Up <Keyboard>/w
    Down <Keyboard>/s
    Left <Keyboard>/a
    Right <Keyboard>/d
```

Save the asset and make sure the action names and binding groups are correct. RebindX uses binding GUIDs, so do not manually depend on an array index in your own code.

3.2 2. Add a Rebind Manager

Create an active GameObject, add `RebindManager`, and assign the Input Action Asset. One manager can own one asset and one active profile. Multiple managers are supported; this is useful for local multiplayer or separate input contexts.

The manager enables the managed asset and loads overrides during startup. If you use a generated controls wrapper, assign it through the corresponding manager configuration or call `SetControls` from code. Keep the manager alive while settings rows are active.

3.3 3. Choose persistence

The Inspector offers three storage locations:

PersistentDataPath Recommended for player settings and shipped games.

DataPath Useful for development tools; generally not writable in a player build.

Custom An explicitly supplied directory. It must not be empty.

The default file is a versioned JSON file below a `Configs` directory. RebindX creates missing directories when saving.

3.4 4. Add one UI row

Create a settings row GameObject and add `RebindActionUI`. Assign:

1. An `InputActionReference` in **Action**.
2. The intended `RebindManager` explicitly.
3. A binding through the custom Inspector's **Binding** popup.
4. Optional TextMeshPro fields for the binding label, action label, and prompt.
5. Optional overlay and UnityEvent listeners.

At runtime the row can be connected to a button:

```
using MLGWorks.RebindX.Runtime;
using UnityEngine;

public sealed class RebindButton : MonoBehaviour
{
    [SerializeField] private RebindActionUI row;

    public void Begin() => row.StartInteractiveRebind();
    public void Cancel() => row.CancelInteractiveRebind();
    public void Reset() => row.ResetToDefault();
}
```

Important. The UI row needs a valid action reference and binding GUID. If either is missing, RebindX will refuse to rebind rather than guessing a binding.

4 How a rebind works

When `StartInteractiveRebind` is called, RebindX creates a Unity `RebindingOperation`, disables the target action as needed, waits for an eligible control, applies the override, checks conflicts, updates the display, saves, and restores the original action/map/asset enabled state.

Cancellation is safe while idle and during an active operation. The previous effective binding is restored when a rebind is cancelled, times out, is interrupted by device removal, or the row is disabled/destroyed.

4.1 Rebind options

`RebindOptions` controls the accepted input and conflict behavior:

Option	Meaning
`bindingGroup`	Restricts the operation to a control scheme/group.
`controlPathsToMatch`	Allow-list of control paths.
`controlPathsToExclude`	Deny-list of control paths.
`cancelControlPath`	Control that cancels the operation; default is Escape.
`expectedControlType`	Requires a type such as Button or Stick.
`minimumMagnitude`	Ignores weak input below this magnitude.
`timeoutSeconds`	Cancels after the given duration; zero disables timeout.
`cancelWhenDeviceIsRemoved`	Cancels if the active device disappears.
`maximumDuplicateRetries`	Number of retries after a rejected conflict.
`duplicateBindingScope`	Checks the same action map or the entire asset.
`duplicateBindingPolicy`	Legacy Reject/Allow policy.
`duplicateBindingResolution`	Reject, Allow, Replace, or Swap.

Example:

```
row.rebindOptions = new RebindOptions
{
    bindingGroup = "Gamepad",
    expectedControlType = "Button",
    timeoutSeconds = 15f,
    maximumDuplicateRetries = 2,
    duplicateBindingScope = DuplicateBindingScope.EntireAsset
};
row.rebindOptions.controlPathsToExclude.Add("<Gamepad>/leftStick");
```

5 Composite bindings

A composite is one logical binding made of multiple parts, for example a 2D movement binding with Up, Down, Left, and Right. Create composites in Unity's Input Actions editor as usual.

For a whole-composite row, select the composite header. RebindX walks the parts in order. It clears old composite overrides before displaying freshly bound parts; this prevents default paths and stale overrides from appearing together. If the operation is cancelled, the old overrides are restored.

You may also create one row per part. This is useful when the UI has separate controls for each direction. Always ensure each row references the intended binding GUID and manager.

6 Duplicate handling and conflict resolution

RebindX detects collisions within the configured scope and respects control-scheme groups. A binding in an unrelated exclusive scheme is not considered a conflict; ungrouped bindings are treated as global.

Reject	Keep the old binding, report the conflict, and optionally retry.
Allow	Permit duplicate controls. The legacy `duplicateBindingPolicy` setting has precedence for compatibility.
Replace	Keep the newly selected control and remove the conflicting binding's override.
Swap	Exchange the target's previous effective path with the conflicting binding's path.

Subscribe to `duplicateBindingEvent` for a user-facing message and `duplicateResolutionEvent` for the selected resolution. A retry limit of zero rejects immediately. This makes conflict behavior deterministic and suitable for keyboard, gamepad, and multiple-control-scheme settings screens.

7 Profiles

Profiles provide stable IDs, display names, and independent override files. They are useful for keyboard/controller presets, multiple local players, accessibility layouts, or separate control schemes.

```
manager.CreateProfile("keyboard", "Keyboard and Mouse");
manager.CreateProfile("gamepad", "Controller");
manager.SwitchProfile("gamepad");
manager.RenameProfile("gamepad", "Xbox Controller");
```

The active profile cannot be deleted. Profile IDs must be safe file identifiers. Switching saves the current profile, clears active overrides, and loads the selected profile. Profile metadata is stored beside the configured binding file.

8 Persistence and recovery

The JSON store writes a versioned envelope containing:

- the format version;
- an asset or profile identity;
- Unity's binding-overrides JSON.

Before replacing a save, RebindX writes a temporary file. A malformed file is not silently applied: it is moved to a timestamped `.corrupt-*.json` quarantine file and a `CorruptData` result is returned. A file for a different asset/profile returns `AssetMismatch`; unsupported future versions return `UnsupportedVersion`.

The service result is available to callers:

```
BindingOverrideResult result = manager.SaveRebinds();
if (!result.Succeeded)
    Debug.LogWarning($"Could not save controls: {result.Code} {result.Message}");
```

Possible result codes include `Success`, `NoData`, `InvalidAsset`, `InvalidPath`, `AssetMismatch`, `UnsupportedVersion`, `CorruptData`, and `IoFailure`.

9 Resetting and changing assets

Reset one row with `ResetToDefault`. Reset the whole manager with `ResetRebinds`; this removes active overrides and deletes the persisted file. Explicit `SaveRebinds` and `LoadRebinds` are useful for Apply/Discard workflows.

For dynamically loaded assets:

```
public void UseAsset(InputActionAsset asset)
{
    rebindManager.SetActionAsset(asset);
}
```

The previous asset is disabled, the new asset is enabled, and the configured overrides are loaded. Passing null is rejected. Generated wrappers can be provided through `SetControls`; the manager disposes the wrapper it owns when it is replaced.

10 Device-aware displays and prompts

The UI assembly includes `IDeviceBindingDisplayProvider`. The default provider normalizes layouts into `Keyboard`, `Mouse`, `Gamepad`, `Joystick`, `Touchscreen`, `XR`, `Pen`, or `Unknown`. It also produces stable glyph keys such as `keyboard.enter`, `mouse.left_button`, and `gamepad.button_south`.

Use `deviceBindingDisplayEvent` to connect the result to a sprite atlas, glyph font, or localized prompt system:

```
row.deviceBindingDisplayEvent.AddListener((source, device, glyph, prompt) =>
{
    glyphView.SetGlyph(glyph);
    promptLabel.text = prompt;
});
```

For a project-specific naming scheme, implement the provider:

```
public sealed class MyGlyphProvider : IDeviceBindingDisplayProvider
{
    public BindingDeviceKind GetDeviceKind(string layout, string path)
        => layout.Contains("Gamepad") ? BindingDeviceKind.Gamepad : BindingDeviceKind.
            Unknown;

    public string GetGlyphKey(string layout, string path)
        => path == "<Gamepad>/buttonSouth" ? "xbox-a" : "generic-control";

    public string GetPrompt(string layout, string path, string type = null)
        => "Press the controller button";
}
row.bindingDisplayProvider = new MyGlyphProvider();
```

11 Accessibility and UI events

Use `rebindAccessibilityEvent` to announce states such as waiting for input, cancelled, timed out, or device removed. Use it with a screen reader, large-text status label, haptic feedback, or audio cue.

Other useful events include binding-display updates, start/stop rebind notifications, duplicate conflict notifications, timeout notifications, and device-aware display updates. Overlay objects are active only during an operation and are hidden on every cleanup path.

Localization can be handled by listening to these events and passing the prompt or status key through the project's localization system. This keeps the core binding logic independent from a particular UI language implementation.

12 Cloud saves and custom integration

12.1 Custom persistence

Implement `IBindingOverrideStore` when storage is not a local JSON file. The interface receives the active `InputActionAsset`; it does not own the manager or the asset lifecycle.

```
public sealed class CloudOverrideStore : IBindingOverrideStore
{
    private string json;

    public BindingOverrideResult Save(InputActionAsset asset)
    {
        json = asset.SaveBindingOverridesAsJson();
        // Queue json for your authenticated cloud service.
        return BindingOverrideResult.Success("Queued for cloud upload.");
    }

    public BindingOverrideResult Load(InputActionAsset asset)
    {
        if (string.IsNullOrEmpty(json))
            return BindingOverrideResult.NoData("No cloud data available.");
        asset.LoadBindingOverridesFromJson(json);
        return BindingOverrideResult.Success("Loaded from cloud.");
    }

    public BindingOverrideResult Delete()
    {
        json = null;
        // Delete the remote profile in your backend here.
        return BindingOverrideResult.Success("Cloud profile deleted.");
    }
}
```

Assign the store before saving or loading. For asynchronous cloud APIs, queue the operation and return a result that accurately describes whether the request was accepted; apply downloaded JSON on the main thread because Unity Input System objects are Unity-owned state.

12.2 Custom paths

Implement `IRebindPathProvider` when the file name or directory comes from a platform service. Always return a stable, writable path, keep player-specific data outside source control, and validate profile identifiers before incorporating them into a path.

12.3 Custom asset ownership

Use `IInputActionAssetProvider` when your project creates or destroys assets dynamically. The provider should clearly define who owns disposal and should restore enabled state consistently. Avoid having several systems independently enable or disable the same asset.

13 Recommended production architecture

For a small project, a manager and a row per binding are enough. For a larger project, use this separation:

```
Settings screen
  -> RebindActionUI rows
  -> RebindManager facade
       -> RebindSession
       -> IBindingOverrideStore
       -> IRebindPathProvider
       -> IInputActionAssetProvider
```

Inject manager references instead of searching the scene. Use one manager per asset/profile context, make cloud synchronization explicit, and display save failures to the player or telemetry system.

14 Testing and release checklist

Before shipping, verify:

- ordinary keyboard, mouse, and gamepad bindings;
- composite headers and individual composite parts;
- cancellation, timeout, disable, destruction, and device removal;
- duplicate rejection, retry limits, Allow, Replace, and Swap;
- separate control schemes and multiple profiles;
- reset, save, load, missing file, corrupt file, mismatch, and unsupported version;
- a read-only or unavailable save location;
- scene reload, domain reload, and multiple managers;
- accessibility announcements and localized prompts;
- cloud conflict policy and offline behavior.

Run the included EditMode and PlayMode tests in Unity Test Runner. Add project-specific PlayMode tests for actual devices and UI navigation; synthetic rebinding operations do not always reproduce every platform's native event processing.

15 Troubleshooting

The default key is displayed after a reset.
A reset intentionally removes the override, so Unity displays the original default binding. Start a new rebind to create a fresh override.

Several rows show the same key.
Check duplicate scope and resolution policy. Use Reject for exclusive controls or Replace/Swap for a clear one-control-per-action layout.

My saved settings do not load.
Check the active profile, file path, asset identity, and result code. An asset mismatch is a safety feature, not a parsing failure.

The file is marked corrupt.
Inspect the quarantined file for diagnostics, then let RebindX create a clean save. Do not load arbitrary JSON without validating its source.

The rebind prompt never finishes.
Check binding groups, excluded paths, expected control type, minimum magnitude, and whether the device is connected.

The UI does not compile.
Install TextMeshPro and Localization, or reference only the core assembly and build a custom UI.

Multiple Utils versions conflict.
Keep one compatible Utils assembly in the project, update all assets together, or distribute a coordinated package dependency.

16 API summary

Type/API	Purpose
`RebindManager`	Unity-facing owner and facade for an input asset and persistence.
`RebindActionUI`	Interactive row for one binding or composite part.
`RebindOptions`	Input filters, timeout, device removal, and conflict policy.
`RebindProfile`	Stable profile ID and display name.
`IBindingOverrideStore`	Replaceable persistence backend.
`JsonBindingOverrideStore`	Versioned, asset-scoped local JSON storage.
`InMemoryBindingOverrideStore`	Non-file storage for tests and temporary contexts.
`IRebindPathProvider`	Replaceable file path resolution.
`IInputActionAssetProvider`	Replaceable asset/wrapper ownership.
`IDeviceBindingDisplayProvider`	Device kind, glyph key, and prompt mapping.
`BindingOverrideResult`	Explicit persistence outcome and diagnostic message.

17 Versioning and support

Treat saved override files as user data. Do not delete them during an application update unless the asset identity or profile migration requires it. If the input asset changes substantially, preserve the old file for recovery and provide a user-facing Reset Controls action.

For support requests, record the RebindX version, Unity version, Input System version, active profile, result code, and whether the issue affects local or cloud persistence. Avoid logging complete player-identifying data or unrelated input settings.

18 Final quick-start checklist

1. Install Input System and optional UI dependencies.
2. Create an Input Action Asset with maps, actions, bindings, and composites.
3. Add and configure one `RebindManager`.

4. Choose PersistentDataPath for player settings.
5. Add `RebindActionUI` rows and assign binding GUIDs with the Inspector.
6. Decide Reject, Allow, Replace, or Swap conflict behavior.
7. Add profiles if more than one layout or player is needed.
8. Connect device display, accessibility, and persistence events.
9. Test cancellation, reset, duplicate controls, corrupt saves, and device removal.
10. Ship the final PDF and source documentation with your asset or internal integration guide.